

NOTE:

- These are not exact midterm questions. These questions are intended to review the material.

1 Loop Invariants**Algorithm 1** Partition algorithm

```

1: function PARTITION( $A$ , low, high)
2:    $x \leftarrow 1$ 
3:    $y \leftarrow \text{length}(A)$ 
4:   for  $i \leftarrow 1$  to  $y + 1$  do
5:     if  $A[i] \leq \text{low}$  then
6:       SWAP( $A, i, x$ )                                ▷ Swaps  $A[i]$  and  $A[x]$ 
7:        $x \leftarrow x + 1$ 
8:     else if  $A[i] \geq \text{high}$  then
9:       SWAP( $A, i, y$ )                                ▷ Swaps  $A[i]$  and  $A[y]$ 
10:       $y \leftarrow y - 1$ 
11:       $i \leftarrow i - 1$ 
12:     end if
13:   end for
14: end function

```

1. What is the loop invariant?

At the start of each iteration of the loop from line 4 to 12, the following conditions hold:

- All elements in $A[1 \dots x - 1]$ are $\leq \text{low}$.
 - All elements in $A[y + 1 \dots \text{length}(A)]$ are $\geq \text{high}$.
 - All elements in $A[x \dots i - 1]$ are $> \text{low}$ and $< \text{high}$.
2. Prove the loop invariant. Use the termination condition to prove the correctness of the overall algorithm.

Initialization: Before the loop starts, $x = 1$ and $y = \text{length}(A)$. Therefore, the subarrays $A[1 \dots 0]$ and $A[y + 1 \dots y]$ do not exist, and the L.I. vacuously holds true.

Maintenance: Assume the L.I. holds true at some arbitrary iteration. There are 3 cases possible.

- (a) **Case 1:** $A[i] \leq \text{low}$
Swapping the element that is $\leq \text{low}$ into position x and incrementing x means

that statement (1) in the L.I. will hold at the next iteration. According to the L.I., $low < A[x] < high$. Therefore, moving element $A[x]$ into position $A[i]$ and then incrementing i will cause statement (3) of the L.I. to hold true at the next iteration. Since y was not changed at all, statement (2) of L.I. will also hold.

(b) **Case 2:** $A[i] \geq high$

Statements (1) and (2) hold by similar arguments made in Case 1. Since i is decrementing in the branch statement, but incremented by the loop, i will remain the same. Since we did not change the contents of the subarray $A[x \dots i - 1]$ during the loop body, then statement (3) of the L.I. will hold at the next step.

(c) **Case 3:** $low < A[i] < high$

In this case, nothing is done except increment i . Since the contents of the entire array don't change, statements (1) and (2) hold at the next iteration (since they have nothing to do with i). The condition itself shows that $A[i]$ validates the condition of statement (3), therefore after incrementing (3) will hold true at the next iteration.

Termination:

At termination, $i = y + 1$. Substituting this fact into the L.I. tells us

- All elements in $A[1 \dots x - 1]$ are $\leq low$.
- All elements in $A[y + 1 \dots \text{length}(A)]$ are $\geq high$.
- All elements in $A[x \dots y]$ are $> low$ and $< high$.

Putting these together: $A[1 \dots x - 1] \leq low < A[x \dots y] < high \leq A[y + 1 \dots A]$.

This means that the array has been partitioned, because it is separated into two parts by two partitions low and high.

2 Reasoning with Short Justification

For each of the following statements, indicate whether the statement is true or false and provide a brief justification.

- a. For two non-negative functions $f(n)$ and $g(n)$, if $f(n) = O(g(n))$, then $\log(f(n)) = O(\log(g(n)))$.

Solution: This statement is **False**. We can disprove it with a counter-example:
 $f(n) = 2^{\frac{1}{n}}$, $g(n) = 2^{\frac{1}{n^2}}$

- b. If we permute an array of length n randomly (by selecting a permutation from any possible permutation uniformly at random), the average running time of INSERTION-SORT on the permuted array will be $\Theta(n \log n)$.

Solution: This statement is **False**. If the permutation of the numbers in the array is random, the inner for loop will iterate $\frac{j}{2}$ times on average, and the overall runtime will still be $\sum_{j=2}^n \frac{j}{2} = \Theta(n^2)$.

- c. Consider sorting the array $A = \langle 9, 5, 7, 6, 3, 4, 1, 2, 8 \rangle$ using RANDOMIZED-QUICKSORT. If we know that 5 and 7 are compared, but 5 and 8 are *not* compared during the course of the algorithm, we can conclude that 6 and 8 are not compared.

Solution: This statement is **True**. From the first two facts, we can infer that 7 becomes the pivot before 5 and 8. Since 5 and 7 are compared, 6 cannot become the pivot before 7. Since 7 becomes the pivot before both 6 and 8, we can conclude that 6 and 8 are never compared.

3 Create an Algorithm:

We are given an array D , which is an array of n distinct strings, sorted in lexicographic order. We are also given a procedure $\text{COMPARE-STRINGS}(x, y)$, which will compare two strings x and y in $\Theta(1)$ time, and return true if x comes before y , and false if y comes before x in lexicographic order. We are also given an array C , which contains $n - 1$ of the n strings in D , but C is not sorted. We would like to develop an efficient algorithm that will find the missing string in C .

General Idea

The FIND-MISSING algorithm identifies a missing element in a sorted list by leveraging a binary search mechanism. It compares elements of the input array C against a reference array D which contains all elements expected in C , while C is being sorted into the same order as D . If C contains all of the elements of D up to point i , we would expect $C[i] = D[i]$. So, if it is, we know the missing element is to the *right*. Otherwise, it is to the *left*. We continue to search recursively until there is only one item left.

Pseudocode:

Algorithm 2 Find the Missing Element

```

1: procedure FIND-MISSING( $C, D$ )
2:    $p \leftarrow 1$ 
3:    $r \leftarrow \text{length}(D)$ 
4:   while  $p \leq r$  do
5:      $m \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
6:      $flag \leftarrow \text{TRUE}$ 
7:     for  $i = p$  to  $r$  do
8:       if COMPARE-STRINGS( $C[i], D[m]$ ) == 0 then
9:          $flag \leftarrow \text{FALSE}$ 
10:      end if
11:    end for
12:    if  $flag$  then
13:      return ( $D[m]$ )
14:    end if
15:     $C, j \leftarrow \text{PARTITION}(C, p, r, D[m])$  ▷ Partition C around D[m]
16:    if  $j < m$  then
17:       $r \leftarrow m - 1$  ▷ Missing element is to the left
18:    else
19:       $p \leftarrow m + 1$  ▷ Missing element is to the right
20:    end if
21:  end while
22: end procedure
23: function PARTITION( $C, p, r, x$ ) ▷ Provided for convenience, may be assumed in
    presentations.
24:    $i \leftarrow p$ 
25:   for  $k = p$  to  $r$  do
26:     if COMPARE-STRINGS( $C[k], x$ ) ≤ 0 then
27:       Swap  $C[i]$  with  $C[k]$ 
28:        $i \leftarrow i + 1$ 
29:     end if
30:   end for
31:   return  $C, i$ 
32: end function

```

Runtime Analysis

Let $T(n)$ denote the runtime of FIND-MISSING on an instance of size n . The PARTITION procedure works in linear time as it goes through the array exactly once, performing only $\Theta(1)$ -time work. Since we use the middle element of D as the pivot, we know that PARTITION will split the array into two equal halves (within a difference of 1). After running PARTITION, FIND-MISSING recurses on only one side of the array. Putting these altogether, we obtain:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(n).$$

Applying the master theorem, we have the root as $f(n) = n$ and the number of leaves as $n^{\log_2 1} = 1$. Since $1 < n$, this is Case 3, thus we have $T(n) = \Theta(n)$.

Proof of Correctness:

Loop Invariant:

- $D[1...p-1]$ are all elements in $C[1...p-1]$
- $D[r+1...length(C)]$ are all elements in $C[r+1...length(C)]$

Initialization: Before the loop, $p = 1$ and $r = length(C)$. Since the subarrays $C[1...0]$ and $C[length(C)...length(C)-1]$ don't exist, then the L.I. is vacuously true.

Maintenance: Assume the L.I. is true at some arbitrary iteration. Upon each iteration, the midpoint, m , is calculated. We first check directly if the pivot we chose is actually in the array C , and if it's not we correctly return the word as missing.

Otherwise, the array C is partitioned based on the pivot chosen as $D[m]$. Since array D is sorted, there must be $m-1$ words that are "less than" $D[m]$. We know the partition algorithm correctly returns the index of the pivot, which was originally chosen as element $D[m]$. Therefore, $D[m] = C[j]$. This must mean that $m-1$ words occur before element $C[j]$, or else one of these words is missing. This is true in the case where $j < m$, so a word is missing in the left half of C . This means all the words in the right half of D , i.e. $D[m..length(C)]$ are also in the right half of C . Therefore, updating $r = m-1$ ensures that the L.I. is true at the next iteration. A similar argument applies if $j = m$, just flipped.

Termination: There are two cases when the algorithm terminates.

- **Case 1:** $D[m]$ not found in $C[p...r]$

If the pivot we chose is not in the portion of the string that hasn't been verified yet, then $D[m]$ is the missing word, and the algorithm correctly returns it.

- **Case 2:** $p = r + 1$

Substituting this fact into the L.I. gives $D[1..r]$ are all elements in $C[1...r]$, and $D[r+1...length(C)]$ are all elements in $C[r+1...length(C)]$. Since the lengths of C and D differ by one, this must mean that $D[length(D)]$ is the missing word, so the algorithm correctly returns it.